

Gödel's Incompleteness Theorem

Philip Eugen Nesbakken

February 20, 2026

Introduction

The early twentieth century was a period of profound transformation in the study of the foundations of mathematics. This was marked by intense debates about the nature of truth, the reliability of formal systems and the limits of mathematical formalisation. In 1931, Kurt Gödel first published his Incompleteness Theorems, that fundamentally reshaped our understanding of mathematics and logic. Let us first examine the historical context of when Gödel published his paper.

1 Historical Context

Hilbert's Program and the Quest for Certainty

In the late 19th and early 20th centuries, mathematicians such as David Hilbert sought to establish a firm foundation for mathematics. Hilbert's program aimed to formalize all of mathematics into a complete and consistent axiomatic system, where every mathematical statement could be either proved or disproved using a finite set of rules. This vision was motivated by the desire to eliminate the paradoxes and inconsistencies that had arisen in set theory and other areas of mathematics [5].

Hilbert's program relied on two key assumptions:

1. **Completeness:** Every mathematical statement should be decidable within the formal system.
2. **Consistency:** The system should be free of contradictions, which means that no statement could be proved and disproved at the same time.

These goals were part of a broader effort to secure the certainty of mathematical knowledge, which had been called into question by developments such as Georg Cantor's work on infinite sets and the paradoxes discovered by Bertrand Russell and others [3].

The Development of Formal Logic

The development of formal logic in the late 19th and early 20th centuries provided the tools necessary for Hilbert's program. Bertrand Russell's *Principia Mathematica* laid the groundwork for formalizing mathematics in a rigorous way [sep-frege]. These efforts demonstrated that mathematical reasoning could be reduced to symbolic manipulation, raising hopes that all mathematics could be systematized in a similar manner.

However, this optimism was challenged by the emergence of paradoxes and the realization that some mathematical questions might be inherently undecidable. For example, the Liar Paradox and Russell's Paradox exposed limitations in naive approaches to formalizing truth and set theory [1].

Gödel's Incompleteness Theorems

In 1931, Kurt Gödel published his paper *On Formally Undecidable Propositions of Principia Mathematica and Related Systems*, in which he demonstrated that any formal system capable of expressing basic arithmetic is either incomplete or inconsistent. Specifically, Gödel showed:

1. **First Incompleteness Theorem:** In any consistent formal system that is sufficiently powerful to express arithmetic, there exist true statements that cannot be proved within the system.
2. **Second Incompleteness Theorem:** Such a system cannot prove its own consistency.

These results directly contradicted Hilbert's program revealing that the goals of completeness and consistency were impossible within a formal system.

We will examine the proof in Gödel's 1931 paper; however, less well known is his *first* version of his incompleteness proof. We will first look at Juliette Kennedy's description of the first proof Gödel laid out before his 1931 paper, as described in his 1934 Princeton Lectures.[4]

2 Gödel's Proofs

Gödel Numbering: Formal Construction and Proof

[Gödel Numbering of Symbols] For a formal language L of arithmetic with symbols $(,), \neg, \rightarrow, \forall, 0, S, +, \times, =, x_1, x_2, \dots$, we assign prime numbers:

$$\begin{array}{llll} (= 3, &) = 5, & \neg = 7, & \rightarrow = 9, \\ \forall = 11, & 0 = 13, & S = 17, & + = 19, \\ \times = 23, & = = 27, & x_k = 29 + 2k & \end{array}$$

[Gödel Numbering of Expressions] For an expression $E = s_1 s_2 \dots s_n$ (sequence of symbols), its Gödel number is:

$$E = \prod_{i=1}^n p_i^{s_i}$$

where p_i is the i -th prime number.

[Unique Factorization] Every expression has a unique Gödel number, and given any number $n \in \mathbb{N}$, we can effectively determine whether n encodes an expression and if so, reconstruct the original expression.

Proof. By the fundamental theorem of arithmetic, each number has a unique prime factorization. The exponents in this factorization correspond to symbol codes. The process is computable because:

1. Prime factorization is computable
2. Checking if exponents correspond to valid symbol codes is computable
3. The original string can be reconstructed from the exponents

□

[Arithmetization of Syntax] Define the following primitive recursive functions and relations:

$\text{Term}(n) := n$ is a Gödel number of a term

$\text{Form}(n) := n$ is a Gödel number of a formula

$\text{Free}(n, v) :=$ Variable v occurs free in formula n

$\text{Subst}(n, v, t) :=$ Result of substituting term t for variable v in formula n

Theorem 2.1 (Representability of Syntax). *All syntactic notions (Term , Form , Free , Subst) are representable in $\mathcal{L}$. That is, for each there exists a formula φ such that:*

1. If $R(\bar{n})$ holds, then $\vdash \varphi(\bar{n})$
2. If $\neg R(\bar{n})$ holds, then $\vdash \neg \varphi(\bar{n})$

where $\bar{n}$ is the numeral for n .

Proof Sketch. The key steps are:

1. Show the basic coding functions are primitive recursive
2. Verify $\mathcal{L}$ can represent all primitive recursive functions
3. Construct formulas that mirror the recursive definitions of:
 - Term formation rules
 - Formula formation rules
 - Free variable computation
 - Substitution algorithm
4. Prove by induction that these formulas correctly represent the syntactic notions

□

[Diagonalization Lemma] For any formula $\varphi(x)$ with one free variable, there exists a sentence G such that:

$$\vdash G \leftrightarrow \varphi(G)$$

Proof. Define $\text{diag}(n)$ as the Gödel number of the formula obtained by substituting the numeral for n into the formula with Gödel number n (if possible). This function is computable and thus representable in .

Let $\theta(x) := \varphi(\text{diag}(x))$ and let $m = \theta(x)$. Then $G := \theta(m)$ is the desired fixed point. □

[Effectiveness of Gödel Numbering] The Gödel numbering satisfies:

1. Injective: Different expressions have different numbers
2. Computable: Both encoding and decoding are algorithmic
3. Representable: All syntactic operations are expressible in Peano Arithmetic

Intuitionistically Acceptable?

NOTE: Assumed knowledge is MA531: Mathematical Logic

“I have explained the [...] undecidab(ility) in a given formal system in the lectures I gave in Princeton in 1934. [...] However in consequence of the philosophical prejudices of our times [...] a concept of objective mathematical truth as opposed to demonstrability was viewed with greatest suspicion and widely rejected as meaningless.” [2]

According to this letter that Gödel sent to Balas, he already had a proof of the Incompleteness Theorem before his 1931 paper that he describes in his 1934 Princeton Lectures. He rejected this first “intuitionist” proof of the Incompleteness Theorem due to the prejudices at the time. However, this is a good place to start to analyse exactly how Gödel reaches his conclusions in his 1931 paper.

The high level idea of the proof is

1. $\mathfrak{N}$: There exists an arithmetic system $\mathfrak{N}$ such that $\mathfrak{N}$ is sound. For this paper, we will assume this is the language and system of number theory.
2. **Provability**: There exists a set of $\mathfrak{N}$ -provable sentences in the language of $\mathfrak{N}$.
3. **Subset**: $\mathfrak{N}$ -provable sentences are a strict subset of all sentences in $\mathfrak{N}$.

If $\mathfrak{N}$ is sound, meaning intuitively that a sentence is either true or false, not both, then the set of all sentences that can be proved or disproved in $\mathfrak{N}$ is a strict subset of $\mathfrak{N}$. This means there exists sentences that cannot be proved or disproved in $\mathfrak{N}$.

Let us examine the proof in more detail. (This will be proven in the examination of the 1931 Final Version of Gödel's Proof)

We will firstly assume the *Fixed Point Theorem*.

Theorem 2.2 (Fixed Point Theorem). *Let T be a formal theory that includes a sufficient amount of arithmetic, and let $\phi(x)$ be a wff in the language of T with one free variable. Then there exists a sentence ψ s.t*

$$T \vdash \psi \leftrightarrow \phi(\ulcorner \psi \urcorner)$$

where $\ulcorner \psi \urcorner$ is the Gödel number of ψ .

Using the

Theorem

Every primitive recursive function is representable in Peano Arithmetic (PA). That is, for every primitive recursive function $f(x_1, \dots, x_n)$, there exists a formula $\varphi_f(x_1, \dots, x_n, y)$ in the language of arithmetic such that:

1. If $f(a_1, \dots, a_n) = b$, then

$$\text{PA} \vdash \varphi_f(\overline{a_1}, \dots, \overline{a_n}, \overline{b})$$

- 2.

$$\text{PA} \vdash \forall y (\varphi_f(\overline{a_1}, \dots, \overline{a_n}, y) \rightarrow y = \overline{b})$$

Here, $\overline{n}$ denotes the standard numeral for the natural number n .

Proof Sketch

The proof proceeds by induction on the structure of primitive recursive functions.

1. Basic Functions

- **Zero Function:** $Z(x) = 0$
Represented by: $\varphi_Z(x, y) := y = 0$
- **Successor Function:** $S(x) = x + 1$
Represented by: $\varphi_S(x, y) := y = x + 1$
- **Projection Function:** $P_k^n(x_1, \dots, x_n) = x_k$
Represented by: $\varphi_{P_k^n}(x_1, \dots, x_n, y) := y = x_k$

All these are trivially representable in PA.

2. Closed under Composition

Suppose $f(x_1, \dots, x_n) = h(g_1(\vec{x}), \dots, g_m(\vec{x}))$ and that $g_1, \dots, g_m, h$ are representable by formulas φ_{g_i} and φ_h .

We define:

$$\varphi_f(\vec{x}, y) := \exists z_1 \cdots \exists z_m \left(\bigwedge_{i=1}^m \varphi_{g_i}(\vec{x}, z_i) \wedge \varphi_h(z_1, \dots, z_m, y) \right)$$

This uses the closure of PA under bounded quantifiers and conjunctions. Thus, f is representable.

3. Closed under Primitive Recursion

Let

$$f(0, \vec{x}) = g(\vec{x}), \quad f(n+1, \vec{x}) = h(n, f(n, \vec{x}), \vec{x})$$

Assume g and h are representable by φ_g and φ_h .

Define $\varphi_f(n, \vec{x}, y)$ to express that y is the result of applying the recursion up to n . We define:

$$\varphi_f(n, \vec{x}, y) := \exists w \left(\text{Seq}(w, n+1) \wedge \text{Init}(w, \vec{x}) \wedge \text{Rec}(w, n, \vec{x}) \wedge \text{Last}(w, y) \right)$$

Where:

- $\text{Seq}(w, n+1)$: w is a sequence of length $n+1$
- $\text{Init}(w, \vec{x})$: $\varphi_g(\vec{x}, w_0)$
- $\text{Rec}(w, n, \vec{x})$: For all $i < n$, $\varphi_h(i, w_i, \vec{x}, w_{i+1})$
- $\text{Last}(w, y)$: $w_n = y$

Sequences can be coded as numbers using Gödel's β -function or pairing functions, and PA can reason about these encodings.

Conclusion

By induction on the definition of primitive recursive functions, every such function is representable in Peano Arithmetic.

□

Theorem (Fixed Point Theorem / Diagonal Lemma)

Let T be a formal theory that includes sufficient arithmetic (such as Peano Arithmetic), and let $\varphi(x)$ be a formula in the language of T with one free variable. Then there exists a sentence ψ such that:

$$T \vdash \psi \leftrightarrow \varphi(\ulcorner \psi \urcorner)$$

where $\ulcorner \psi \urcorner$ is the Gödel number of ψ .

Proof

Define a function $\text{Sub}(x, y)$ that takes the Gödel number x of a formula with one free variable and the Gödel number y of a term, and returns the Gödel number of the formula obtained by substituting the term with code y for the variable in the formula with code x . Since substitution is a primitive recursive function, it is representable in arithmetic.

Let $\theta(x) := \varphi(\text{Sub}(x, x))$. That is, θ is a formula that, given input x , substitutes x into itself and applies φ to the result.

Let $e := \ulcorner \theta \urcorner$, the Gödel number of the formula θ . Now define:

$$\psi := \theta(e) = \varphi(\text{Sub}(e, e))$$

Then by construction:

$$T \vdash \psi \leftrightarrow \varphi(\ulcorner \psi \urcorner)$$

since $\ulcorner \psi \urcorner = \text{Sub}(e, e)$, which is computable and representable in T .

□

Why is this first version of the proof not enough in Gödel's own words?

Why the First Version Was Not Enough

Gödel's initial proof, while insightful, was not sufficient for several reasons. In his own words, Gödel expressed concerns about the philosophical climate of the time, which was heavily influenced by intuitionism and formalism. These schools of thought were skeptical of the notion of objective mathematical truth, favoring instead a focus on constructibility and provability. Gödel's first proof relied on the concept of truth, which was viewed with suspicion and often rejected as meaningless by his contemporaries. This made the proof less acceptable to the broader mathematical community.

Moreover, the first version of the proof was not as rigorous or general as the one presented in his 1931 paper. Gödel recognized the need to frame his results in a way that was independent of philosophical assumptions about truth. By shifting the focus to provability within formal systems, he was able to present a more universally acceptable argument that did not rely on contentious metaphysical claims.

The Final Version of Gödel's Proof

In his 1931 paper, Gödel refined his approach by focusing on the concept of formal provability rather than truth. This allowed him to construct a sentence that essentially states, "I am not provable." The key steps in this proof are as follows:

1. **Gödel Numbering:** Gödel assigned a unique number to each symbol, formula, and proof in the formal system. This encoding allowed him to translate statements about the system into arithmetic statements.
2. **Diagonalization:** Using the Fixed Point Theorem, Gödel constructed a sentence G that is equivalent to the statement " G is not provable." This self-referential sentence plays a crucial role in the proof.
3. **Incompleteness:** If the system were complete, it would be able to prove G . However, if G were provable, it would lead to a contradiction because G asserts its own unprovability. Therefore, G must be true but unprovable, demonstrating the incompleteness of the system.

This final version of the proof not only established the incompleteness of formal systems but also demonstrated the limitations of Hilbert's program. By showing that no consistent formal system can prove its own consistency, Gödel revealed the inherent limitations of mathematical formalization.

The Transition to the Formalisation of Truth

Gödel's First Incompleteness Theorem is one of the most profound results in mathematical logic. It states that any consistent formal system capable of expressing basic arithmetic is incomplete, meaning there exist true statements that cannot be proved within the system. This section provides a detailed proof of the theorem.

3 Preliminaries

Before proceeding with the proof, we need to establish some key definitions and concepts.

[Formal System] A formal system $\mathcal{F}$ consists of a formal language, a set of axioms, and a set of inference rules. The system is said to be *consistent* if no contradiction can be derived from its axioms.

[Gödel Numbering] Gödel numbering is a method of assigning a unique natural number to each symbol, formula, and proof in a formal system. This allows statements about the system to be encoded as arithmetic statements. See section above for more in detail.

[Provability Predicate] Let $\mathcal{F}$ be a formal system. The provability predicate $\text{Prov}(x)$ is a formula in the language of $\mathcal{F}$ that expresses the statement " x is the Gödel number of a provable sentence in $\mathcal{F}$."

4 Proof of the First Incompleteness Theorem

We now present the proof of Gödel's First Incompleteness Theorem.

Theorem 4.1 (Gödel's First Incompleteness Theorem). *Let $\mathcal{F}$ be a consistent formal system capable of expressing basic arithmetic. Then there exists a sentence G in the language of $\mathcal{F}$ such that:*

1. G is true.
2. G is not provable in $\mathcal{F}$.

Proof. The proof proceeds in several steps:

Step 1: Gödel Numbering

Assign a unique Gödel number to each symbol, formula, and proof in $\mathcal{F}$. This allows us to encode statements about $\mathcal{F}$ as arithmetic statements.

Step 2: Constructing the Provability Predicate

Define the provability predicate $\text{Prov}(x)$ in $\mathcal{F}$, which expresses that " x is the Gödel number of a provable sentence in $\mathcal{F}$." This predicate is constructed using the axioms and inference rules of $\mathcal{F}$.

Step 3: Diagonalization

Using the technique of diagonalization, construct a sentence G such that:

$$G \leftrightarrow \neg \text{Prov}(\ulcorner G \urcorner),$$

where $\ulcorner G \urcorner$ is the Gödel number of G . This sentence G essentially states, "I am not provable in $\mathcal{F}$."

Step 4: Analyzing G

We now analyze the properties of G :

1. Suppose G is provable in $\mathcal{F}$. Then, by the definition of G , $\neg \text{Prov}(\ulcorner G \urcorner)$ is also provable. This leads to a contradiction because $\mathcal{F}$ would simultaneously prove G and its negation, violating consistency.
2. Therefore, G cannot be provable in $\mathcal{F}$. Since G asserts its own unprovability, it must be true.

Conclusion

We have shown that G is true but not provable in $\mathcal{F}$. This demonstrates that $\mathcal{F}$ is incomplete, as there exists a true statement that cannot be proved within the system. $\square$

5 Implications

Gödel's First Incompleteness Theorem has profound implications for the foundations of mathematics. It shows that no formal system capable of expressing basic arithmetic can be both consistent and complete. This result undermines Hilbert's program, which sought to establish a complete and consistent axiomatization of all mathematics.

6 Connections to Church and Turing

Gödel's Incompleteness Theorems laid the groundwork for the theory of computation, influencing Alonzo Church's and Alan Turing's work on decidability and computability. Here, we formalize these links.

6.1 Church's Thesis and Undecidability

[Effectively Computable] A function is *effectively computable* if there exists an algorithm (a finite mechanical procedure) to compute its value for any input.

Theorem 6.1 (Church's Thesis (1936)). *The class of effectively computable functions coincides with the class of recursive functions (Gödel's primitive recursive + μ -recursion) and, equivalently, the λ -definable functions (Church's λ -calculus).*

[Undecidability of First-Order Logic] Church and Turing independently showed that the *Entscheidungsproblem* (Hilbert's decision problem for first-order logic) is unsolvable. This follows from:

- Gödel's encoding of proofs as numbers (arithmetization).
- The existence of a formal system $\mathfrak{N}$ where provability is representable but not decidable.

6.2 Turing Machines and Incompleteness

[Turing Machine (1936)] A *Turing machine* is a theoretical device manipulating symbols on a tape according to a finite table of rules. It formalizes the intuitive notion of computation.

Theorem 6.2 (Turing's Universality). *The halting problem for Turing machines is undecidable: no algorithm can determine whether an arbitrary Turing machine halts on a given input.*

Sketch: Reduction to Gödel's Theorem. 1. Encode Turing machines as Gödel numbers in $\mathfrak{N}$.

2. If the halting problem were decidable, $\mathfrak{N}$ could resolve the provability of G (the "I am not provable" sentence), violating incompleteness.

□

6.3 Synthesis: Incompleteness and Uncomputability

The developments by Gödel, Church, and Turing between 1931 and 1936 form a coherent picture of the limitations of formal systems and computation. Though approached from different angles—arithmetization, function theory, and mechanical computation—their results are deeply interrelated and converge on the same conclusion: *not all mathematical truth is capturable by formal or algorithmic means*.

1. Gödel: Incompleteness through Arithmetization

Gödel showed that in any consistent, sufficiently expressive formal system F (such as Peano Arithmetic), there exists a statement G such that:

$$F \vdash G \leftrightarrow \neg \text{Prov}_F(\ulcorner G \urcorner)$$

If F is consistent, G is true but unprovable in F . This established that *truth in arithmetic transcends formal provability*. His technique relied on arithmetizing syntax and constructing self-referential statements through diagonalization.

2. Church and Turing: Uncomputability via Function Theory and Machines

Independently, Church (via λ -calculus) and Turing (via Turing machines) formalized the notion of an *effective procedure* or algorithm. They showed that:

- The **Entscheidungsproblem**—whether there exists an algorithm to decide the validity of any first-order logical formula—is undecidable.
- The **Halting Problem**—whether a Turing machine halts on a given input—is undecidable.

Their results demonstrated that no general algorithm can solve all instances of certain well-defined mathematical problems.

3. Equivalence of Computability Models

The models of computation developed by Church (recursive functions and λ -calculus) and Turing (Turing machines) were shown to be extensionally equivalent. This led to the widely accepted **Church–Turing Thesis**:

A function is effectively computable if and only if it is computable by a Turing machine (or equivalently, is recursive or λ -definable).

4. Convergence of Results

The deeper insight is that Gödel’s arithmetization of syntax and Turing’s encoding of machines both yield formal systems where:

- There exist true but unprovable statements (Gödel)
- There exist computable problems with no algorithmic solution (Turing, Church)

The *same* techniques—diagonalization, self-reference, and encoding—are used in each proof. For example, Turing’s proof of the Halting Problem mirrors Gödel’s method: it constructs a machine that, when asked whether it halts, tries to simulate its own behavior, leading to contradiction.

5. Summary Table

Concept	Gödel (1931)	Church / Turing (1936)
Formalization	Arithmetization of syntax	λ -calculus / Turing machines
Limitation	Incompleteness of formal systems	Undecidability of computation
Key Result	True but unprovable statements	No algorithm for Halting Problem
Mechanism	Self-reference, diagonalization	Self-reference, diagonalization
Implication	Not all truth is provable	Not all problems are solvable

6. Final Insight

These results collectively show that mathematics and logic are subject to intrinsic limitations: there exist truths that are neither provable within formal systems nor decidable by mechanical computation. The 1930s revealed that the boundaries of human knowledge are not only philosophical but formal and computational.

Conclusion

Kurt Gödel’s Incompleteness Theorems marked a turning point in the foundations of mathematics, demonstrating that no formal system capable of expressing elementary arithmetic can be both complete and consistent. His meticulous arithmetization of syntax and the construction of self-referential statements not only resolved key questions posed by Hilbert’s program but also inaugurated a new era in logic and computation.

By proving that there exist true but unprovable statements, Gödel revealed that mathematical truth transcends formal deduction. This insight was soon echoed and expanded by the work of Church and Turing, who formalized the notion of computability and established the undecidability of foundational problems such as the Halting Problem and the Entscheidungsproblem. Their contributions, though methodologically distinct, are deeply intertwined with Gödel’s results—sharing techniques of diagonalization, encoding, and self-reference.

Together, the theorems of Gödel, Church, and Turing unveiled the intrinsic limitations of formal reasoning and algorithmic processes. These limitations are not flaws of mathematical

systems, but rather inherent boundaries of any attempt to fully capture truth through finite formal means. In this sense, the incompleteness and undecidability theorems not only answer long-standing questions, but also invite humility and wonder in the pursuit of mathematical knowledge.

As we reflect on these results nearly a century later, their influence endures, in philosophy of mathematics, theoretical computer science, artificial intelligence, and beyond. They remain a profound reminder that the most rigorous formal systems still reside within a wider conceptual truth.

References

- [1] Jc Beall. “The Liar Paradox”. In: *Stanford Encyclopedia of Philosophy* (2018). URL: <https://plato.stanford.edu/entries/liar-paradox/>.
- [2] Solomon Feferman. “Arithmetization of Metamathematics in a General Setting”. In: *Fundamenta Mathematicae* 49.1 (1960). ISSN: 0016-2736.
- [3] Thomas Jech. “Set Theory”. In: *Stanford Encyclopedia of Philosophy* (2021). URL: <https://plato.stanford.edu/entries/set-theory/>.
- [4] Juliette Kennedy. *Gödel’s Incompleteness Theorems*. Cambridge Elements in Philosophy and Logic. Cambridge: Cambridge University Press, 2021. ISBN: 978-1108973878.
- [5] Richard Zach. “Hilbert’s Program”. In: *Stanford Encyclopedia of Philosophy* (2019). URL: <https://plato.stanford.edu/entries/hilbert-program/>.